different distributions of their consistency problems, but it is probably quite representative that the larger scopes of consistency are the most difficult to get right.

5.5 Feedback

The system should continuously inform the user about what it is doing and how it is interpreting the user's input. Feedback should not wait until an error situation has occurred: The system should also provide positive feedback, and it should provide partial feedback as information becomes available. For example, the way to write the German letter ü on many keyboards involves first typing the umlaut, ¨, and then typing the character that is to go under the two dots. Some systems provide no visible feedback as the first part of the character is typed, leading many novice users to conclude that the system does not know how to deal with umlauts. A better design would show the umlaut and then change the cursor in some way to indicate that the system was waiting for the second part of the character.

System feedback should not be expressed in abstract and general terms but should restate and rephrase the user's input to indicate what is being done with it. For example, it is a good idea to give a warning message in case the user is about to perform an irreversible action, such as overwriting a file (see Section 5.9). Assume that the user is about to copy a file to another disk and that the copy operation would overwrite a file with the same name. The worst feedback (except none, of course) would be to state that a file was about to be overwritten, without giving its name. Better feedback would include the name of the file, and even better feedback would include attributes of the two files, such as file type and modification date, to help the user understand whether the copy operation was just replacing an old copy with a newer copy of the same file or whether the file being overwritten was in fact a completely different file that happened to have the same name.

Different types of feedback may need different degrees of persistence in the interface [Nielsen 1987c]. Some feedback is only rele-

vant for the duration of a certain phenomenon, and can thus have low persistence, going away when it is no longer needed. For example a message stating that the printer is out of paper should be removed automatically once the problem has been fixed. Other feedback needs to have medium persistence and stay on the screen until the user explicitly acknowledges it. An example in this category would be a message stating that the user's output had been rerouted to another printer because of some problem with the printer specified by the user. Finally, a few types of feedback may be so important that they require high persistence, remaining a permanent part of the interface. An example might be the indication of remaining free space on a hard disk.

Response Time

Feedback becomes especially important in case the system has long response times for certain operations. The basic advice regarding response times has been about the same for many years [Miller 1968; Card *et al.* 1991]:

- 0.1 second is about the limit for having the user feel that the system is reacting instantaneously, meaning that no special feed-

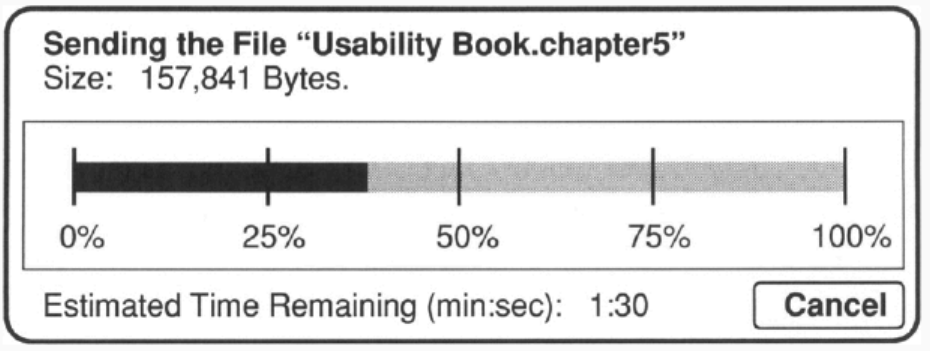

Figure 14 *Percent-done indicator for a hypothetical file-transfer program. The design not only provides feedback expressed in the user's terms (the name of the file) and with respect to the progress of the transfer, it also provides an easy way out (cf. Section 5.6) in case the user gets tired of waiting or discovers that the wrong file is being transferred.*

remain within the available window. The fact that computers can

remain within the available window. The fact that computers can be *too* fast indicates the need for user-interface changes, like anima-

For operations where it is unknown in advance how much work has to be done, it may not be possible to use a percent-done indicator, but it is still possible to provide running progress feedback in terms of the absolute amount of work done. For example, a system searching an unknown number of remote databases could print the name of each database as it is processed. If this is not possible either, a last resort would be to use a less specific progress indicator in the form of a spinning ball, a busy bee flying over the screen, dots printed on a status line, or any such mechanism that at least indicates that the system is working, even if it does not indicate what it is doing.

For reasonably fast operations, taking between 2 and 10 seconds, a true percent-done indicator may be overkill and, in fact, putting one up would violate the principle of display inertia (avoiding flash changes on the screen so rapidly that the user cannot keep pace or feels stressed). One could still give less conspicuous progress feedback. A common solution is to combine a "busy" cursor with a rapidly changing number in small field in the bottom of the screen to indicate how much has been done.

System Failure

Informative feedback should also be given in case of system failure. Many systems are not designed to do so and simply stop responding to the user when they go down. Unfortunately, no feedback is almost the worst possible feedback since it leaves users to guess what it wrong. Systems can be designed for graceful degradation, enabling them to provide some feedback to users even when they are mostly down.

As an example, consider feedback to users of an automated teller machine (ATM). On February 13, 1993, all 1,200 ATMs belonging to a major bank in New York City refused to perform any user transactions for a period of four hours due to a bug in a software

going on and hoped that other machines might be working. Since it would be unrealistic to expect a 1,200 node distributed computer